[이름 입력]

Server Developer Intern

justinnn980@gmail.com
[전화번호 입력]
GitHub | 배포 API
[학교 / 전공 / 졸업 상태 입력]

제출 전 확인: 이름, 전화번호, 학력·졸업 상태를 실제 정보로 교체해 주세요.

PROFILE

서비스의 전체 흐름을 완성해 본 서버 개발자

Java와 Spring Boot로 요구사항을 API와 데이터 구조로 바꾸고, 인증·권한·데이터 저장·성능 개선·자동 테스트·배포·운영 확인까지 연결했습니다. 단순히 기능이 동작하는 데서 멈추지 않고, 다른 사용자의 데이터 접근과 중복 요청, 반복 조회, 배포 후 장애 확인처럼 실제 서비스에서 생길 문제를 먼저 정의하고 테스트로 검증하는 개발 방식을 익히고 있습니다.

SKILLS

기술

서버 개발	Java 21, Spring Boot, Spring MVC, REST API, Gradle
데이터	Spring Data JPA, Hibernate, PostgreSQL, H2, 관계 설계, 인덱스
인증·성능	Spring Security, JWT, BCrypt, Redis 캐시, 중복 요청 방지
품질·운영	JUnit, MockMvc, Mockito, GitHub Actions, Docker, Render, Actuator

PROJECT

Study Tracker API

2026.06 - 현재 | 개인 프로젝트 | 서버 전체 구현

학습 Todo를 관리하는 API 서버입니다. 회원가입과 로그인부터 사용자별 Todo 관리, PostgreSQL 저장, Redis 활용, 자동 테스트·빌드, Render 배포와 상태 확인까지 한 프로젝트 안에서 구현했습니다.

49	JWT	60초	1회
자동 테스트	로그인·재발급·로그아웃	통계 캐시 유지 시간	관계 조회 쿼리 검증

- 회원가입·로그인과 토큰 재발급·로그아웃을 구현하고, 비밀번호는 단방향 암호화해 저장했습니다.
- Todo 조회·수정·삭제 때 Todo 번호뿐 아니라 로그인한 사용자까지 함께 확인해 다른 사용자의 데이터 접근을 차단했습니다.
- 통계 조회 결과를 Redis에 60초 저장하고, Todo가 바뀌면 해당 사용자의 캐시를 지워 정확성과 반복 조회 효율을 함께 확보했습니다.
- 사용자와 Todo를 함께 조회하는 상황에서 여러 추가 쿼리가 발생하는 것을 확인하고, 한 번의 쿼리로 가져오는 방식으로 바꾼 뒤 테스트로 검증했습니다.
- GitHub에 코드를 올리면 49개 테스트와 빌드를 자동 실행하도록 구성하고 Docker·Render로 배포했습니다.

PROBLEM SOLVING

문제 해결 경험

01

사용자 데이터 보호

Todo 번호만으로 조회하면 다른 사용자의 데이터가 노출될 수 있다고 판단했습니다. 모든 조회·수정·삭제를 'Todo 번호 + 로그인 사용자' 조건으로 제한했고, 타인의 목록이 보이지 않으며 조회·수정·삭제가 모두 거절되는지 API 테스트로 확인했습니다.

02

반복 통계 조회 개선

같은 통계를 볼 때마다 데이터베이스에서 전체·완료 개수를 다시 계산하는 비용을 문제로 정했습니다. 결과를 Redis에 60초 저장하고 데이터 변경 시 즉시 삭제했습니다. 두 번째 조회가 저장된 값을 사용하고, 변경 뒤에는 새 값이 계산되는지 자동 테스트로 검증했습니다.

03

중복 생성 요청 방지

더블 클릭이나 네트워크 재시도로 같은 Todo가 두 번 생성될 수 있다고 보았습니다. 사용자·작업·요청 식별값을 묶어 Redis에 5분간 저장하고 최초 요청만 처리했습니다. 같은 요청을 다시 보내면 409 응답이 오는지 API 테스트로 확인했습니다.

04

관계 조회 쿼리 최적화

사용자 목록 뒤에 각 사용자의 Todo를 읽으면 사용자 수만큼 추가 쿼리가 생길 수 있음을 테스트에서 재현했습니다. 필요한 관계를 첫 쿼리에서 함께 가져오도록 바꿔, 두 사용자 기준 여러 번 발생하던 조회를 1회로 줄였음을 통계로 검증했습니다.

QUALITY & OPERATIONS

테스트·배포·운영

49개 자동 테스트

회원가입, 로그인, 토큰 만료, Todo 기능, 타인 데이터 접근, 캐시 저장·삭제, 중복 요청, 데이터 저장 계층, 서버 상태 확인을 검증했습니다.

배포 이후까지 확인

GitHub Actions로 테스트와 빌드를 자동화하고, Render에서 PostgreSQL·Redis를 연결했습니다. 상태 확인 API와 장애 점검 순서를 문서로 남겼습니다.

GROWTH

성장 방향

요구사항을 빠르게 코드로 옮기는 사람을 넘어, 트래픽과 장애 상황에서도 설명 가능한 구조를 만드는 서버 개발자로 성장하고 싶습니다. 현재는 Java·Spring 기반 기능을 테스트·배포·운영까지 연결하는 연습을 반복하고 있습니다. 다음 단계로 Kotlin, 비동기 메시지 처리, 부하 테스트, 지표 수집과 대시보드를 프로젝트에 적용해 '동작한다'를 넘어 '얼마나 안정적으로 동작하는지'를 수치로 설명하겠습니다.

LINKS

링크

GitHub: github.com/justinnn980/study-tracker-api
배포 API: study-tracker-api-7stm.onrender.com
API 문서: Swagger UI | 상태 확인: Health Check